

AGENTIC AI FOR CYBERSECURITY · MODULE 5

LangGraph

Modelling agents as graphs over shared state — and deciding, deliberately, how much autonomy a security system should be granted.

Four parts · two notebooks · a reference template · one mini-project

Prerequisite: the LangChain module

PART 1

From Chains to Graphs

The vocabulary you will use all module: state, nodes, edges, conditional routing, tools, memory — and the decision loop underneath every agent.

MODULE MAP

- **From Chains to Graphs**
- Autonomy, Control & Guardrails
- Build a Stateful Agent
- Lab & Mini-Project

From following recipes to making decisions

A chain executes the path you wrote. An agent selects a path at run time — that is the whole difference.

1

Concepts

State, nodes, edges, conditional routing, tools, memory.

2

Autonomy & control

The autonomy spectrum and the four default guardrails.

3

Build

A triage agent that branches, calls a tool and remembers.

4

Your build

Concept-check lab, then your own stateful security agent.

WHAT CHANGES WHEN THE SYSTEM CHOOSES

Property	Chain (previous module)	Graph agent (this module)
Execution path	Identical for every input	Selected at run time from the state
Termination	Guaranteed — the pipeline ends	Must be designed: bound the loop, guarantee a path to END
Debugging question	“Did each step work?”	“Why did it choose that step?”
Blast radius	Produces text	Can take actions through tools

Why a graph rather than a chain?

Same small, testable functions — but the route through them adapts to the data.

CHAIN · ONE FIXED PATH

```
graph LR; S1[Step 1] --> S2[Step 2]; S2 --> S3[Step 3];
```

Every input traces the same route. Predictable, testable, inflexible.

Termination is free.

GRAPH · A ROUTE CHOSEN PER INPUT

```
graph LR; C[Classify] --> R{route}; R --> E[Enrich]; R --> Cl[Close]; E --> End[END]; Cl --> End;
```

Suspicious alerts detour through enrichment; benign ones do not.

Capability	Why it matters in security work	What it costs you
Branch	Different alert classes need genuinely different handling	A routing decision that can be wrong, quietly
Loop	Investigations refine: gather, reassess, gather again	Non-termination unless you bound it explicitly
Revisit a node	Re-score after new context arrives	State that must stay coherent across visits

Why a graph and not a chain

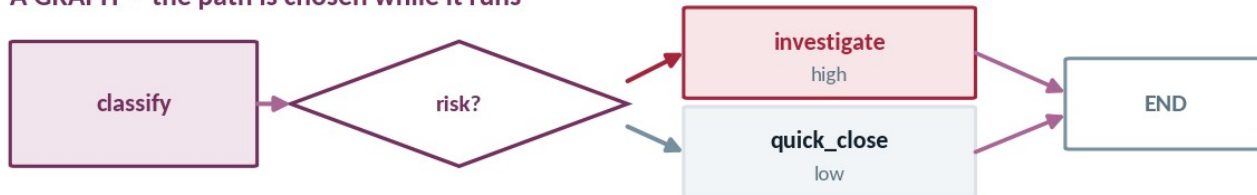
A chain's path is fixed before you run it. A graph's path is chosen while it runs — that is the whole difference.

A CHAIN · the path is fixed before you run it



same steps, same order, every time

A GRAPH · the path is chosen while it runs



What the graph adds

state

a shared dict every node reads and updates

nodes

plain functions — they return an update, they do not mutate

edges

the wiring, including conditional ones

checkpointer

memory, keyed by thread_id

END

the exit — a graph without one will not compile

This is a control-flow library, not a new kind of AI. Deciding what to do next, remembering what you already did, and knowing when to stop are ordinary programming problems. LangGraph makes them explicit rather than hiding them inside a prompt — which is what makes an agent debuggable.

Five words is the whole vocabulary

state · nodes · edges · checkpointer · END.

If you can explain those five, you can read any LangGraph program, including the ones you will meet in module 8.

State, nodes and edges: the whole vocabulary

State is the contract between your nodes. Keep it minimal, typed and explicit.

The three primitives, in code

```
from typing import TypedDict, Optional

class TriageState(TypedDict):          # STATE
    alert: str
    verdict: Optional[str]
    context: Optional[str]
    recommendation: Optional[str]

def classify(state: TriageState) -> dict:  # NODE
    return {"verdict": label(state["alert"])}
```

Term	Definition — and the rule that goes with it
State	The shared, typed record every node reads and writes. Rule: name only what the agent truly needs.
Node	A function: state in, a partial update out. Rule: return only the fields you changed.
Edge	The wire from one node to the next. END is the exit. Rule: every path must be able to reach it.

Partial updates: A node returns {"verdict": "suspicious"} — not the whole state. LangGraph merges it. Returning the full dict is the most common beginner bug.

Why type it: The TypedDict is documentation your editor can check and a review artefact a colleague can read in ten seconds.

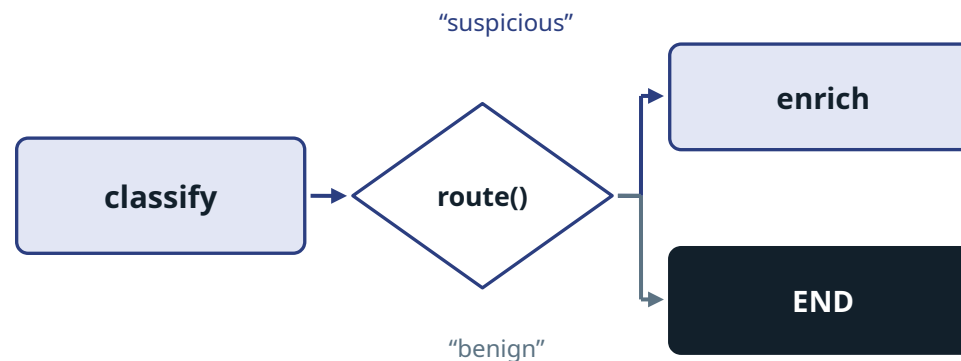
Conditional edges: the moment it becomes an agent

A router reads the state and returns a key. The key names the next node — or END.

A router is an ordinary function returning a branch key

```
def route(state: TriageState) -> str:
    if state["verdict"] == "suspicious":
        return "enrich"
    return "close"

graph.add_conditional_edges(
    "classify", route, {"enrich": "enrich", "close": END},
)
```



Routing basis	Example	Reliability concern
A deterministic rule over state	verdict == "suspicious"	Predictable; only as good as the classifier that set the field
A model-produced label	The classify node asks an LLM for a category	Off-vocabulary labels; always provide a default branch
A confidence threshold	score > 0.8 → act; otherwise escalate	Threshold choice is a policy decision, so document it

Tools and memory: reaching out, and holding on

A tool fetches ground truth the model cannot know. A checkpointer makes the agent stateful across turns.

A tool node — the agent reaches real data

```
def enrich(state: TriageState) -> dict:
    ip = extract_ip(state["alert"])
    rep = reputation_lookup(ip)      # the tool call
    return {"context": f"{ip}: {rep}"}
```

A checkpointer + thread_id — the agent remembers

```
from langgraph.checkpoint.memory import MemorySaver

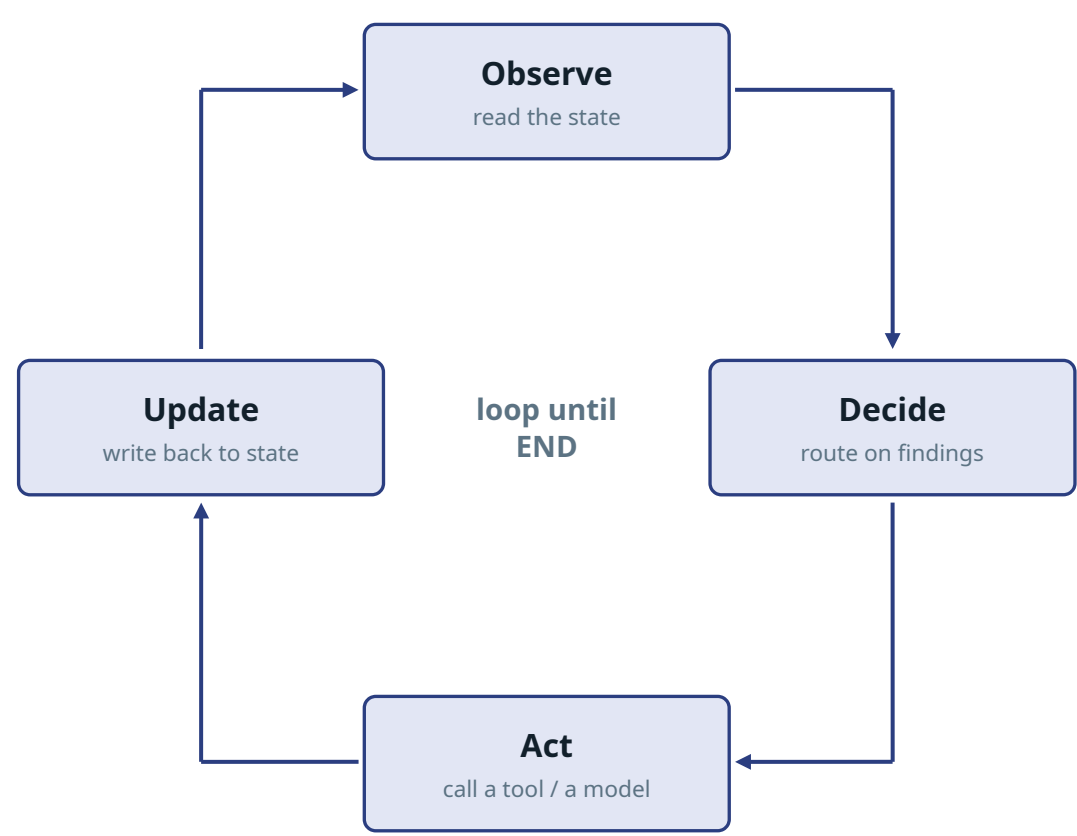
app = graph.compile(checkpointer=MemorySaver())
app.invoke(inp, {"configurable": {"thread_id": "soc-42"}})
```

	What it gives the agent	Who decides when	Security consequence
Tool	Current, external, verifiable facts — a CVE record, an IP reputation, a scan result	The agent, from the state — which is why tools need scoping	Tool output re-enters the context as untrusted text
Memory	Continuity across turns, keyed by thread_id	You, by supplying (or withholding) the thread id	Persisted state is data at rest: it can leak, and it can be poisoned

Same thread, same memory: Re-run with thread_id "soc-42" and the agent recalls turn one. Use a new id and it starts clean — which is also how you isolate one investigation from another.

Every agent is one loop, made explicit

Observe → decide → act → update. The graph's contribution is that you can watch it happen.



Bound the loop — a design requirement, not a nicety

```
def route(state) -> str:
    if state["steps"] >= MAX_STEPS:
        return "halt" # guaranteed exit
    return "investigate" if state["open_leads"] else "report"
```

Unbounded loop risk	Control
Runaway cost	A step counter in the state, checked by the router
Repeated tool calls	Rate limits and idempotent tool design
Silent stall	A wall-clock deadline; log every transition

PART 2

Autonomy, Control & Guardrails

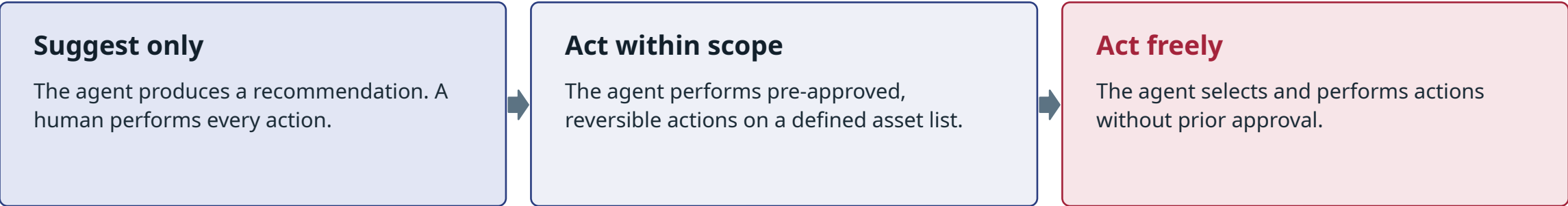
The professional judgement at the centre of agentic security: more autonomy is more useful and more dangerous. Here are the defaults, and how to inspect what the agent decided.

MODULE MAP

- From Chains to Graphs
- **Autonomy, Control & Guardrails**
- Build a Stateful Agent
- Lab & Mini-Project

The autonomy spectrum — and where to sit by default

Design rule: grant the least autonomy that still accomplishes the task.



increasing capability →

← increasing reversibility and oversight

Level	Appropriate task	Required control	Question to ask first
Suggest only	Triage, enrichment, drafting an IR summary	Review before any action is taken	Is a human already in this workflow?
Act within scope	Opening a ticket, tagging an asset, querying read-only APIs	An allow-list, an audit log, and reversibility	Can I undo this in one step?
Act freely	Rarely justified in a security context	Hard technical limits — not policy documents	What is the worst action reachable from here?

Test for your own design: Name the worst action your agent can reach. If you cannot say it in one sentence, the scope is too wide.

Four guardrails, on by default

These are not advanced techniques. They are the baseline for any agent you ship.

1

Human in the loop

A person approves any destructive or irreversible step.

2

Bounded loops

A step counter and a guaranteed path to END.

3

Least-privilege tools

Only the access the task actually requires — read-only by default.

4

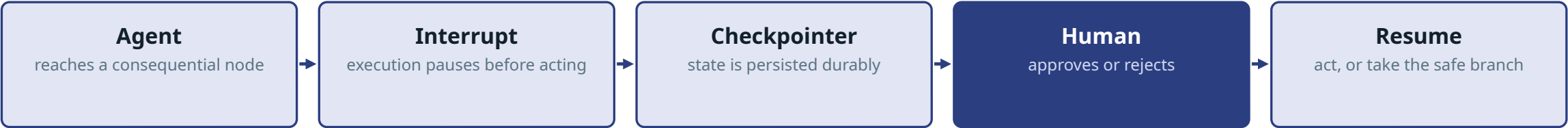
Complete logging

Every node's input, output and branch, retained for audit.

Guardrail	Failure it prevents	How you evidence it	How it degrades
Human in the loop	An irreversible action taken on a wrong verdict	An interrupt in the graph; an approval record	Approval fatigue — volume makes review nominal
Bounded loops	Runaway cost; repeated calls against live systems	A counter in the state; a max-steps test	A limit set so high it never triggers
Least privilege	A minor bug becoming a major incident	Tool scopes and credentials, documented	Scope creep as features are added
Complete logging	An unanswerable “why did it do that?”	A per-node trace you can replay	Logs nobody reads, or logs holding secrets

Human-in-the-loop checkpoints

The agent proposes; the human disposes. LangGraph supports this natively through interrupts.



```
Pause before the consequential node

app = graph.compile(
    checkpointer=MemorySaver(),
    interrupt_before=["quarantine_host"],
)
```

Gate this	Let this run
Blocking an address; quarantining a host	Reading a log; querying reputation
Sending external communication	Drafting a message for review
Anything you cannot undo in one step	Anything idempotent and read-only

Design the approval, not just the pause: Show the reviewer the alert, the verdict, the evidence and the exact action proposed. An approval prompt that says only "Proceed?" trains people to click yes.

Tracing: answering “why did it do that?”

Because every step is an explicit node, a run is a record you can read — and replay.

ONE RUN OF THE TRIAGE AGENT, NODE BY NODE

#	Node	State in (abridged)	State update returned	Next
1	classify	alert: “5 failed SSH from 10.0.0.9”	verdict: “suspicious”	route()
2	route	verdict: “suspicious”	— (no state change)	enrich
3	enrich	alert, verdict	context: “10.0.0.9 — known scanner”	recommend
4	recommend	alert, verdict, context	recommendation: “block + ticket”	END

The zero-dependency version: stream and print

```
for step in app.stream(inputs, config):
    for node, update in step.items():
        print(node, "->", update)
```

Read the trace for	Because
The branch that fired	Mis-routing is the most common agent bug
The tool's raw output	It is untrusted text now inside the context
State at the last node	Wrong answers usually start upstream

Where graph agents earn their keep in security

Four patterns where an auditable path matters as much as the answer.

1

SOC triage
Classify, enrich and recommend — along a path an analyst can audit afterwards.
Our worked example

2

Runbooks as graphs
The documented incident-response procedure and the executable workflow become one artefact.
Drift between doc and practice disappears

3

Branching investigations
Multi-step enquiry that changes direction on findings, with each step recorded.
Loops need bounding

4

Controlled orchestration
Several tools driven in a defined sequence with approval gates.
Extended in the Orchestration module

AND WHERE A GRAPH AGENT IS THE WRONG TOOL

Situation	Why an agent hurts	Use instead
High-volume, well-specified, repetitive work	Non-determinism and per-call cost with no upside	A deterministic pipeline or a rule engine
Hard real-time or safety-critical decisions	Latency is variable and reasoning is not guaranteed	Deterministic logic with an agent for post-hoc review

PART 3

Build a Stateful Agent

Two notebooks: first a bare graph — state, nodes, edges — then the upgrade to an agent that branches, calls a tool and remembers across turns.

MODULE MAP

- From Chains to Graphs
- Autonomy, Control & Guardrails
- **Build a Stateful Agent**
- Lab & Mini-Project

Start with the state — it is the contract

Naming the fields up front is drawing the boxes on a form before anyone fills it in.

```
lecture2_langgraph_state_nodes.ipynb

from typing import TypedDict, Optional

class TriageState(TypedDict):
    alert: str # set by the caller
    verdict: Optional[str] # set by classify
    context: Optional[str] # set by enrich
    recommendation: Optional[str] # set by recommend
```

Field	Written by	Read by	Empty means
alert	the caller	all nodes	invalid input
verdict	classify	route, recommend	not yet classified
context	enrich	recommend	no enrichment ran
recommendation	recommend	the caller	the agent has not concluded

THREE HABITS THAT MAKE A STATE DEFINITION REVIEWABLE

Minimal

Every field earns its place. If no node reads it, delete it.

Optional by default

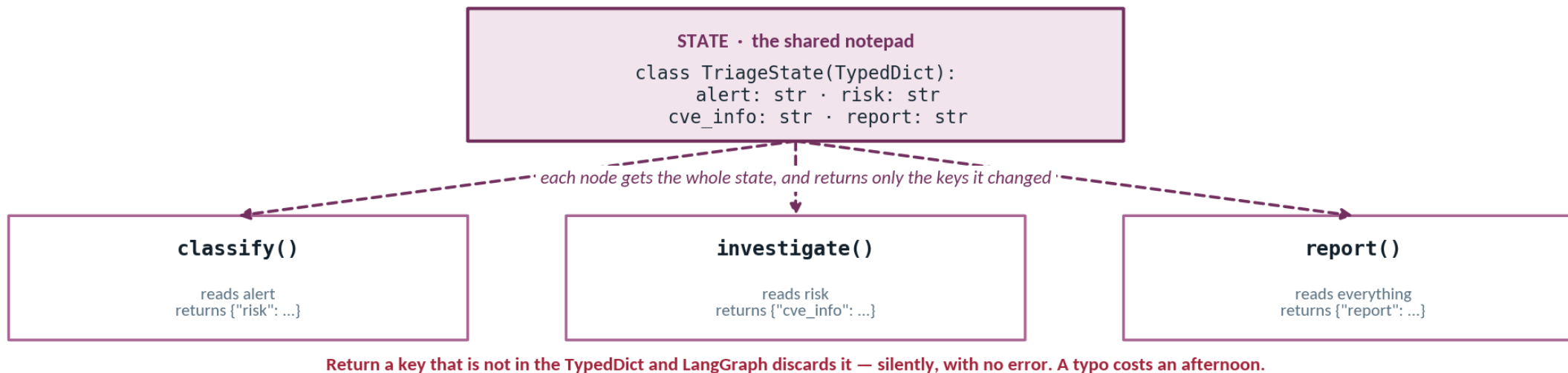
Fields a node has not reached yet are None, not empty strings.

Annotated

A comment naming the writer turns the class into a design document.

The state is the contract

Every node reads the whole state and returns only the keys it changed. Get that wrong and nothing complains.



lecture3_langgraph_branch_tools.ipynb — THE CONTRACT

```

from typing import TypedDict

class TriageState(TypedDict):
    alert: str          # what came in
    risk: str           # "high" / "low"
    cve_info: str
    report: str
  
```

A NODE IS A PLAIN FUNCTION

```

def classify(state: TriageState) -> dict:
    high = any(w in state["alert"].lower()
               for w in ["rce", "ransomware", "critical"])
    return {"risk": "high" if high else "low"}
    # ^ return the update — never mutate state
  
```

Nodes are functions; StateGraph is the wiring

Add nodes, set an entry point, add edges, compile. The graph logic is ordinary Python — no key required.

```
Assemble and compile

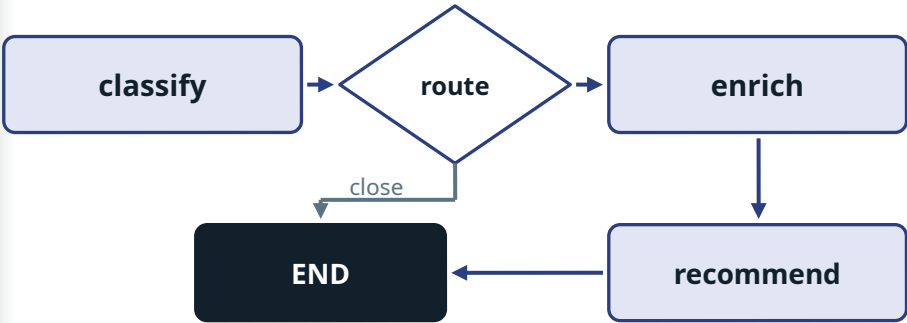
from langgraph.graph import StateGraph, END

graph = StateGraph(TriageState)
graph.add_node("classify", classify)
graph.add_node("enrich", enrich)
graph.add_node("recommend", recommend)

graph.set_entry_point("classify")
graph.add_conditional_edges("classify", route,
                           {"enrich": "enrich", "close": END})
graph.add_edge("enrich", "recommend")
graph.add_edge("recommend", END)

app = graph.compile()
```

THE GRAPH THIS CODE BUILDS

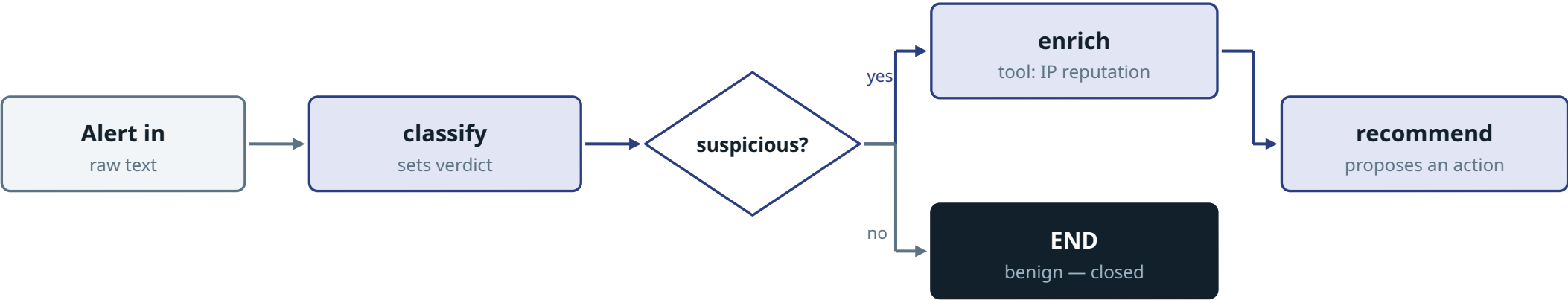


Render it: `app.get_graph().draw_ascii()` prints the structure. Check the picture matches your intent before you debug behaviour.

Build-time error	Symptom	Fix
A node name that no edge targets	The node never executes; no error is raised	Draw the graph and look for orphans
A branch key missing from the mapping	KeyError at run time, on one input class only	Always supply a default branch

Worked example: the triage agent

The graph you extend in the lab and start your project from.



Node	Responsibility	Design note
classify	Label the alert; write verdict to state	Keep the label vocabulary closed and small — the router depends on it
route	Read verdict, return a branch key	Not a node: a plain function, deterministic, and always with a default
enrich	Call the reputation tool; write context	The only node that touches the outside world — scope it read-only
recommend	Propose an action from alert + verdict + context	Proposes. It does not act. Acting is a separate, gated node

Conditional edges and a tool

lecture3_langgraph_branch_tools.ipynb — the router turns a pipeline into an agent, and the tool is how it reaches outside itself.

lecture3_langgraph_branch_tools.ipynb — THE CELL WHERE IT BECOMES AN AGENT

```
# the router: state in, a LABEL out — not a boolean
def route_by_risk(state: TriageState) -> str:
    return "investigate" if state["risk"] == "high" else "quick_close"

# a tool is just a function the graph can call
def lookup_cve(keyword: str) -> str:
    db = {"rce": "CVE-2026-0001: critical RCE; patch immediately.",
          "ransomware": "Multiple CVEs; isolate and restore."}
    for k, v in db.items():
        if k in keyword.lower():
            return v
    return "No specific CVE found; follow standard triage."

# wiring: the router picks a label, the map turns it into a node
b.add_conditional_edges(
    "classify",
    route_by_risk,
    {"investigate": "investigate", "quick_close": "quick_close"})

b.add_edge("investigate", "report")
b.add_edge("quick_close", "report")
b.add_edge("report", END)
graph = b.compile()
```

1 The router returns a label

Not True/False. A string that the mapping turns into a node name — so you can rewire the graph without touching the logic.

2 A tool is an ordinary function

No decorator, no LLM. Just something the graph can call to reach information it does not already have.

3 Return “nothing found”, do not raise

A tool that raises takes the whole run down. One that reports an empty result keeps the graph moving.

4 Run both branches live

Low risk leaves cve_info empty — visible proof that the branch really did skip a node.

Branch, act, remember: three upgrades

lecture3_langgraph_branch_tools.ipynb turns the skeleton into an agent.

1

Branch

A conditional edge, so different inputs take visibly different routes.

Feed it two alerts and watch the paths diverge.

2

Act

A tool the node calls for real information instead of guessing.

Compare the answer with and without enrichment.

3

Remember

A MemorySaver checkpointer plus a thread_id.

Same thread recalls turn one; a new id starts clean.

Memory in practice — the same graph, two threads

```
cfg_a = {"configurable": {"thread_id": "incident-101"}}
cfg_b = {"configurable": {"thread_id": "incident-102"}}

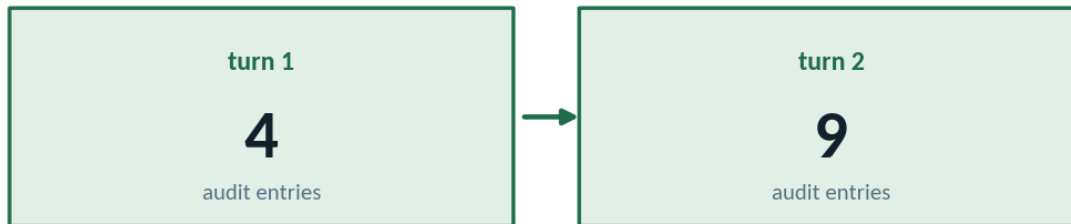
app.invoke({"alert": first_alert}, cfg_a)    # remembered
app.invoke({"alert": second_alert}, cfg_b)  # isolated
```

Notebook	Use it for
lecture2 ...state_nodes	Understanding structure
lecture3 ...branch_tools	Seeing the three upgrades
...reference_template	Starting your project

Memory is a thread, not a global

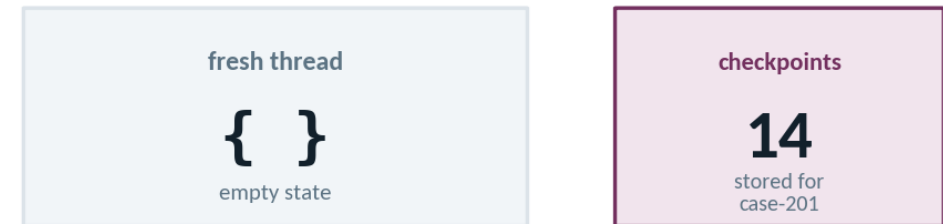
Add a checkpointer and the same `thread\_id` remembers. A different one starts clean. Measured on the reference agent.

SAME thread_id • the state is still there



the trail accumulates — it did not start over

NEW thread_id • a clean slate



The failure mode memory introduces

Turn 2 on a clean URL still reported intel={'verdict': 'malicious'} — left over from turn 1. Memory means stale keys persist unless a node clears them, and nothing warns you.

lecture3_langgraph_branch_tools.ipynb — TWO LINES AND ONE IDEA

```
from langgraph.checkpoint.memory import MemorySaver

trriage_mem = b.compile(checkpointer=MemorySaver())

cfg = {"configurable": {"thread_id": "incident-42"}}
trriage_mem.invoke({"alert": "..."}, cfg) # remembers
trriage_mem.invoke({"alert": "..."}, cfg) # same thread
```

Add a reset node if you add memory

Stale state is harder to spot than missing state — everything looks populated and plausible.

If your mini-project keeps memory across turns, clear the per-item keys at the start of each one.

Hands-on: make the change, watch it flow

Two short exercises now — and the first one is already part of your lab submission.

	Do this	Then observe	What it proves
1	In lecture 2, add a third feature to the extraction node — e.g. <code>has_https</code>	The new field appearing in the state at every later node	You understand partial updates and state merging
2	Add the field to the TypedDict as well	No error where you might expect one — then reason about why	Typed state documents intent; it does not enforce it at run time
3	In lecture 3, craft inputs that take each branch	Different node sequences printed for each input	Routing is data-driven, not fixed
4	Re-run one input twice on the same <code>thread_id</code>	The second run recalls the first	The checkpointer, not the model, is what remembers

Screenshot exercise 1: That change and its output are exactly what the concept-check lab asks for.

If nothing happens: Check that a later node actually reads the field. An unread state field is invisible.

PART 4

The Lab & Your Mini-Project

A concept-check lab as rehearsal, then your own stateful security agent: state, three nodes, a conditional edge and a tool it genuinely calls.

MODULE MAP

- From Chains to Graphs
- Autonomy, Control & Guardrails
- Build a Stateful Agent
- **Lab & Mini-Project**

The concept-check lab: your rehearsal

Run it, change one thing, then explain the five concepts in your own words.

Explain...	A complete answer includes	A common incomplete answer
State	A typed, shared record; nodes return partial updates that are merged	"It stores the data"
Nodes & edges	Functions and wiring; every path must be able to reach END	"Nodes do things and edges connect them"
Conditional routing	A deterministic function over state returning a branch key; needs a default	"It's an if/else"
Tools	External ground truth the model cannot know — and untrusted text on return	"It lets the agent search things"
Memory	A checkpointer plus a thread_id; persistence lives outside the model	"The agent remembers"

Practical half: One small feature change to the lecture-2 notebook, re-run, with a screenshot.

Do the lab first. The project assumes every one of these five is already comfortable.

What the lab actually asks for

Part A is five short answers. Part B is two small changes to a notebook you have already run.

Part	What it asks	Marks	Where credit is lost
A1	What is state, and why a TypedDict	12	Describing it as a variable rather than a shared contract
A2	Node, edge and END	12	Omitting END — a graph without one will not compile
A3	Conditional edges	12	Defining them without giving a real branching example
A4	What a tool is	12	Confusing a tool with the model that chooses it
A5	Checkpointner and thread_id	12	Naming only one of the two — the scoping is the point
B1	Add a third feature to extract_features	25	Changing the function but never running it
B2	One input down each branch	15	Showing only the high-risk path

Expect B1 to look like it did not work. The no-key stub in the lecture 2 notebook always prints “benign”, whatever the features say. Your new feature will appear in the printout and the verdict will not move. That is the notebook, not your code — say so in your answer.

pip install grandalf

``draw_ascii()`` needs it and nothing in the module says so — there is no requirements.txt here at all.

One more thing worth knowing before you build

``extract_features`` does ``url.split("/") [2]`` — hand it a bare hostname with no scheme and it raises `IndexError`. Fine for a demo; not fine for a mini-project that accepts arbitrary input.

Mini-project: your own stateful security agent

Pick a task, define the state, wire three nodes, branch honestly, and call one real tool.

	Requirement	Evidence I look for
1	A security task and a state definition	A TypedDict whose fields map to the task
2	Three or more nodes, single-purpose	Each returns a partial update
3	At least one conditional edge	A router with a default branch
4	At least one tool the agent calls	Called in a run — not merely defined
5	Two or three inputs exercising both branches	Printed traces showing different paths
6	A bounded loop or an acyclic graph	A step counter, or a graph with no cycle

TOOL IDEAS THAT STAY SAFE AND OFFLINE

- A local CVE or IOC lookup over a small file
- A mock scanner returning fixed, plausible results
- A read-only public API, rate-limited
- A log parser over a sample you provide

Scope, and the law: Never point a tool at a system you do not own or have written permission to test. Simulated tools receive full marks — the assessment is your graph, not your target.

Five ways a graph goes wrong

Two of them raise. Three of them do not — and those are the ones that cost you an afternoon.

Three of these five tell you nothing at all

Return a key not in the TypedDict	typo discarded – state unchanged	SILENT
interrupt_before with no checkpointer	the interrupt simply does not happen	SILENT
A stale key from a previous turn	old intel reported as current	SILENT
Router returns a label not in the map	KeyError: 'high'	LOUD
No entry point set	ValueError: Graph must have an entrypoint	LOUD

All five reproduced on langgraph 1.2.9. The loud ones you will fix in a minute; the silent ones are what cost an afternoon.

THE ONE THAT SHOULD WORRY YOU

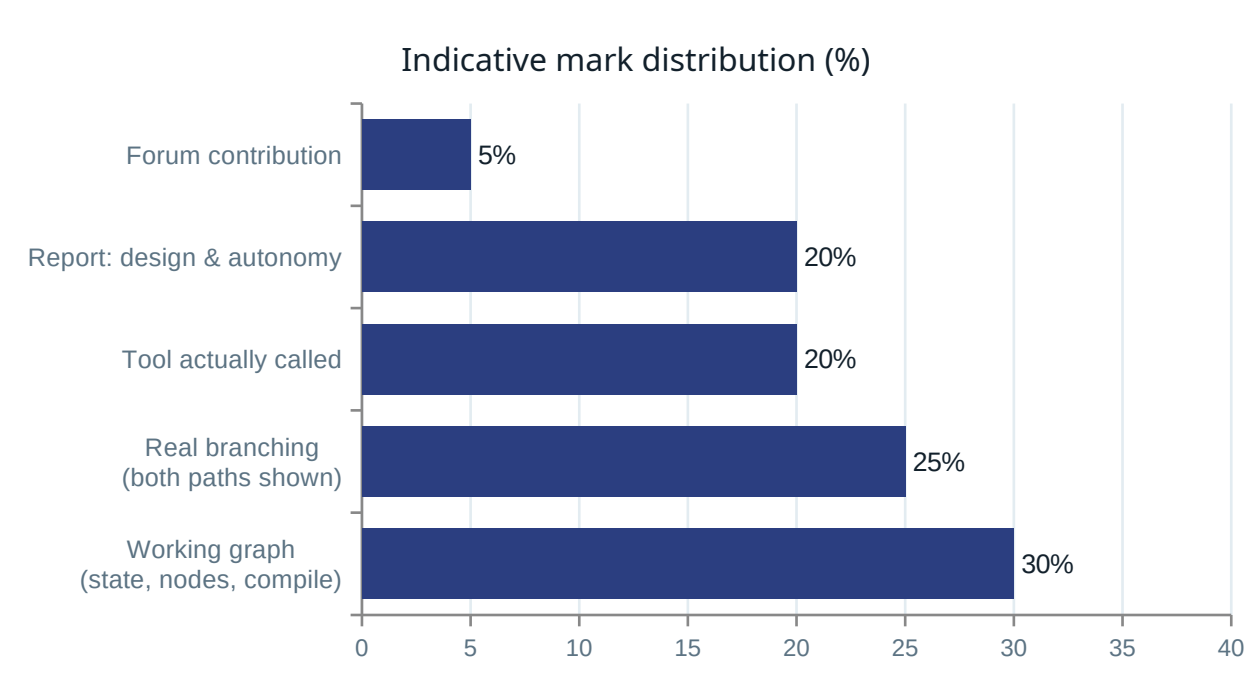
```
# interrupt_before without a checkpointer is a no-op
graph = b.compile(interrupt_before=["block_action"]) # silent
graph = b.compile(interrupt_before=["block_action"],
                  checkpointer=MemorySaver())      # works
```

Permissive systems fail quietly

Print the state between nodes while you develop, and assert that the keys you expect are actually populated. Do not wait for an exception that is never going to come.

Deliverables and where the marks sit

A working graph, real branching, a tool genuinely used — then explain your design choices.



Artefact	Must show
Runnable notebook	Executes top to bottom after a kernel restart
Branch evidence	Traces for 2–3 inputs, different paths
Short report	Why an agent; why this autonomy level
Forum post	One insight or one obstacle

Optional extra: Memory across turns, or a human-in-the-loop interrupt before an acting node.

The report question that separates grades: “What is the worst action your agent can reach, and what stops it?” Answer it in two sentences.

Autonomy, secrets, and what comes next

Two habits to carry forward, and a module where tools stop being something you hand-wire.

Autonomy is a budget, not a target

Grant the least the task requires. Keep a human before any real-world action. That habit is the foundation the Cybersecurity Orchestration module builds on.

Secrets stay out of notebooks

API keys live in .env or Colab Secrets. A key committed to a repository is a security incident, not an untidy commit — and providers do scan for them.

Carry forward	Into
Bounded loops & least privilege	Every agent you build after this
Trace before you theorise	Any non-deterministic system

NEXT MODULE

Model Context Protocol

A standard way to expose tools to any agent — replacing the hand-wiring you did today. Powerful, and a genuinely new attack surface: a tool's description is text the model trusts.

Questions: Post in the module forum — I collect the common ones into the next session's opening.